

torch.from_file#

https://pytorch.org/docs/stable/generated/torch.from_file.html#torch.from_file

Description:

Creates a CPU tensor with a storage backed by a memory-mapped file. If `shared` is `True`, then memory is shared between processes. All changes are written to the file. If `shared` is `False`, then changes to the tensor do not affect the file. `size` is the number of elements in the Tensor. If `shared` is `False`, then the file must contain at least `size * sizeof(dtype)` bytes. If `shared` is `True` the file will be created if needed. Only CPU tensors can be mapped to files.

Function Signature

```
torch.from_file(filename, shared=None, size=0, *,  
dtype=None, layout=None, device=None, pin_memory=False)#
```

Parameters

Keyword Arguments

Parameters

Keyword

`dtype` (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, uses a global default (see `torch.set_default_dtype()`). `layout` (torch.layout, optional) – the desired layout of returned Tensor. Default: `torch.strided`. `device` (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see `torch.set_default_device()`). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types. `pin_memory` (bool, optional) – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default: False.

Code Examples

```
>>> t = torch.randn(2, 5, dtype=torch.float64)
>>> t.numpy().tofile('storage.pt')
>>> t_mapped = torch.from_file('storage.pt', shared=False,
size=10, dtype=torch.float64)
```

Notes & Warnings

NOTE Only CPU tensors can be mapped to files.

NOTE For now, tensors with storages backed by a memory-mapped file cannot be created in pinned memory.

Detailed Documentation

Documentation

Creates a CPU tensor with a storage backed by a memory-mapped file.

If `shared` is `True`, then memory is shared between processes. All changes are written to the file. If `shared` is `False`, then changes to the tensor do not affect the file.

`size` is the number of elements in the Tensor. If `shared` is `False`, then the file must contain at least `size * sizeof(dtype)` bytes. If `shared` is `True` the file will be created if needed.

Only CPU tensors can be mapped to files.

For now, tensors with storages backed by a memory-mapped file cannot be created in pinned memory.

`filename (str)` – file name to map

`shared` (bool) – whether to share memory (whether `MAP_SHARED` or `MAP_PRIVATE` is passed to the underlying `mmap(2)` call)

`size` (int) – number of elements in the tensor

`dtype` (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, uses a global default (see `torch.set_default_dtype()`).

`layout` (torch.layout, optional) – the desired layout of returned Tensor. Default: `torch.strided`.

`device` (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see `torch.set_default_device()`). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types.

`pin_memory` (bool, optional) – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default: False.